Parking Lot Management System Using Python

By

Midhun PS

CSE-C

Roll:NO:5

Abstract

The Parking Lot Management System is a Python-based desktop application designed to digitize and automate the process of vehicle parking and slot allocation. Traditional parking management often relies on manual ticketing, which is prone to human error, double-booking, and inefficient space utilization. This project provides a robust Graphical User Interface (GUI) that allows operators to register vehicles, assign unique slots (e.g., A1, B1), and manage real-time occupancy. The system ensures data integrity through CAPTCHA validation and dynamic slot-availability checking, providing a seamless experience for both the operator and the vehicle owner.

Problem Description

In high-traffic urban areas, parking management faces several challenges:

- **Double Booking:** Without a real-time tracking system, two vehicles might be assigned the same physical slot.
- **Data Retrieval:** Searching for owner information or vehicle details in manual logs is time-consuming.
- **Security:** Unauthorized or unrecorded entries can lead to security breaches.
- **Lack of Structure:** Managing different vehicle types (Cars, Bikes, Buses) without a centralized system leads to disorganized lot usage.

This project addresses these issues by creating a digital ledger where each parking slot is treated as a unique, trackable asset.

Features & Functionality

Slot Management (A1, B1 Logic)

The core "intelligence" of the application lies in its slot validation. Unlike basic forms, this system treats parking slots as unique identifiers.

- **Availability Check:** Before a booking is confirmed, the system parses the selected slot . If "A1" exists in the current session's memory, the system triggers a messagebox.showerror to alert the operator.
- **Slot Recovery:** When a record is deleted, the system automatically removes that slot from the "Occupied" list, making it immediately available for the next customer.

Security Features

- **CAPTCHA Verification:** To prevent accidental or automated "Confirm" clicks, a static CAPTCHA validation is required.
- **Data Integrity:** The "Update" function ensures that if a user changes their slot, the old slot is freed while the new one is locked.

Libraries and Tools Used

The project is built using the Python ecosystem, chosen for its rapid development capabilities and extensive library support.

- **Python (Core Language):** Provides the logic for data handling and object-oriented structure.
- **Tkinter:** The standard Python interface to the Tk GUI toolkit. It is used to create windows, frames, buttons, and input fields.
- **Tkinter.ttk:** Used for themed widgets like Combobox and Treeview to give the application a modern look.
- **PIL (Pillow):** Used for image processing, allowing the integration of icons and branding within the GUI.
- **Messagebox:** A sub-module of Tkinter used for user notifications (Errors, Success alerts, and Confirmations).

System Design and Workflow

The system follows a **Modular Design** where the UI and the Logic are decoupled for easier maintenance.

Workflow:

1. **Entry:** The operator enters the vehicle owner's name and phone number.
2. **Selection:** The operator selects the vehicle type and a specific parking slot (e.g., A1).
3. **Validation:** * The system checks if all fields are filled.
    - The system verifies the CAPTCHA.

    - **The Slot Check:** The system checks the "Occupied List" to ensure the selected slot is not already taken.

4. **Confirmation:** If valid, the data is appended to the Treeview (Table) and the slot is marked as "Occupied."
5. **Management:** The operator can select any row from the table to Update details or Delete the record.

Implementation Details

The core logic of the application is centered around **List Management** and **Event Binding**.

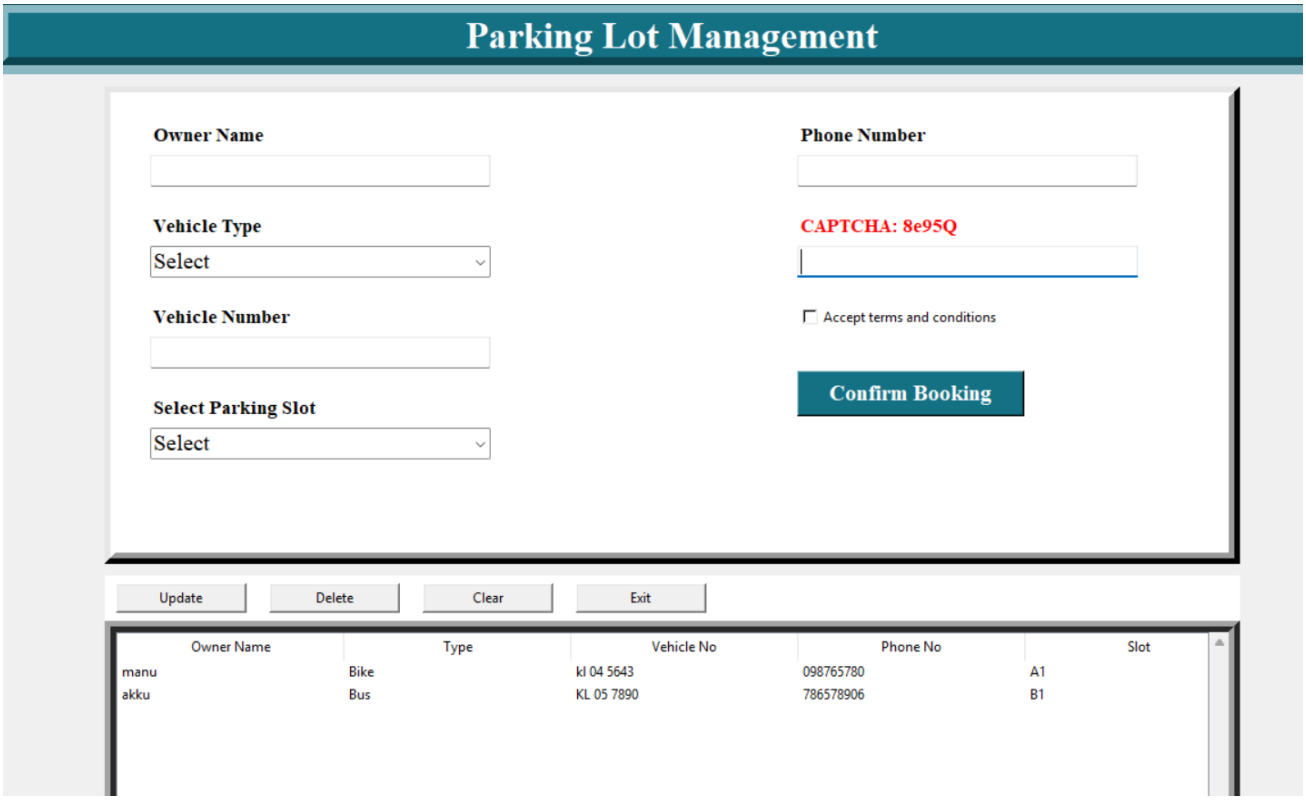
- **Slot Tracking:** A Python list self.occupied_slots stores the strings of taken slots.
- **Treeview Implementation:** The Treeview widget is configured with 5 columns: Owner, Type, Vehicle No, Phone, and Slot. This acts as the "Front-end Database."
- **Event Handling:** The get_cursor function uses .bind to allow users to click a row and have the data automatically populate back into the input fields for editing.

Key Code Logic for Slot Availability:

This simple yet effective check prevents the primary problem of parking mismanagement: overlapping bookings

Result and Output

The final application provides a clean, responsive interface.

- **Success Results:** On successful booking, a "Success" popup appears, the input fields clear, and the table updates instantly.
- **Error Handling:** The system successfully blocks empty submissions, incorrect CAPTCHAs, and double-bookings with descriptive error messages.
- **Scalability:** The current layout supports up to 1000+ records in the Treeview without performance lag.

Sample Output Table Representation: